Manual de usuario

TacticalGrid

Sistema de búsqueda y decisión adversarial en un juego táctico 2D.

1. Requisitos

El programa funciona con Python 3.8 o superior. Se probó con la versión 3.11.

No requiere bibliotecas externas: usa únicamente la biblioteca estándar del lenguaje. Para ejecutar las pruebas automatizadas se necesita pytest, que es opcional.

2. Instalación

Se clona el repositorio y se entra a la carpeta del proyecto.

```
git clone https://github.com/CamiOso/TacticalGrid.git

cd TacticalGrid
```

No hay que instalar dependencias.

3. Ejecución

3.1 Punto de entrada

Todas las pruebas se ejecutan desde un único archivo, start.py, que se encuentra en la raíz del proyecto.

Si se ejecuta sin argumentos, el programa recorre todos los escenarios de la carpeta scenarios.

```
python3 start.py
```

Si se indica un archivo, ejecuta únicamente ese escenario.

```
python3 start.py scenarios/ejemplo_basico.json
```

El comportamiento depende del campo prueba.modo del archivo del escenario. Los modos disponibles son tres.

El modo busqueda resuelve una navegación con alguno de los algoritmos de búsqueda y verifica la solución.

El modo partida simula una partida completa por turnos entre los dos bandos.

El modo adversarial evalúa un estado con Minimax o con alfa-beta y selecciona una acción.

3.2 Opciones de la línea de comandos

Las opciones permiten elegir el algoritmo y sus parámetros sin modificar el código.

La opción algoritmo acepta BFS, DFS, UCS, A_ESTRELLA o BEAM en los escenarios de búsqueda, y MINIMAX o ALFA_BETA en los escenarios adversariales y de partida.

La opción heuristica acepta manhattan, manhattan_pasos, distancia_bfs o zero, y se usa con A* y con la búsqueda en haz.

La opción k define el ancho de la frontera en la búsqueda en haz y debe ser un entero mayor o igual a uno.

La opción profundidad define la profundidad del árbol en Minimax y en alfa-beta, y debe ser un entero mayor o igual a uno.

La opción no-preguntar evita el menú interactivo y usa directamente la configuración del archivo.

Algunos ejemplos de uso.

```
python3 start.py scenarios/terrenos_costosos.json --algoritmo A_ESTRELLA --
heuristica distancia_bfs
```

```
python3 start.py scenarios/divergencia_bfs_ucs.json --algoritmo BEAM --k 4

python3 start.py scenarios/partida_simple.json --algoritmo ALFA_BETA --profundidad 2

python3 start.py scenarios/adversarial_simple.json --algoritmo MINIMAX --profundidad 3
```

Si no se pasa la opción de algoritmo y la ejecución ocurre en una terminal interactiva, el programa muestra un menú numerado para elegir el algoritmo, la heurística y el ancho de la frontera. El archivo del escenario se usa como sugerencia.

3.3 Mapas de desarrollo

Los mapas de desarrollo tienen veinte filas por veinte columnas y tres unidades por bando, de acuerdo con lo solicitado en el enunciado.

El archivo mapa_grande.json se usa para búsqueda y lleva la unidad A1 desde su posición hasta la base del bando B.

El archivo mapa_grande_partida.json se usa para la partida entre tres unidades por bando.

El archivo mapa_grande_adversarial.json se usa para la decisión adversarial sobre un mapa del mismo tamaño.

```
python3 start.py scenarios/mapa_grande.json --algoritmo UCS --no-preguntar
```

3.4 Scripts auxiliares

La carpeta scripts contiene programas de apoyo que se ejecutan desde la raíz del proyecto.

casos_minimos.py reproduce los diez casos mínimos del enunciado.

examples_search.py muestra ejemplos de BFS, DFS, UCS y A*.

examples_adversarial.py muestra ejemplos de Minimax y alfa-beta.

examples_agents.py muestra el uso de los agentes.

experimentos_beam_search.py ejecuta la búsqueda en haz con distintos valores de k.

visualize_search.py genera visualizaciones en HTML.

interactive.py ofrece un recorrido guiado paso a paso.

3.5 Interfaz gráfica

La interfaz se abre con el siguiente comando.

```
python3 -m src.ui.gui
```

Se abre una ventana con tres pestañas: búsqueda, adversarial y partida. En la parte superior se elige el escenario, ya sea desde una lista de la carpeta scenarios o con la opción de abrir un archivo. El tablero se dibuja en el centro y cada pestaña ejecuta su modo correspondiente.

En la pestaña de búsqueda se elige el algoritmo, la heurística y el ancho de la frontera, y el camino encontrado se dibuja sobre el tablero junto con las métricas.

En la pestaña adversarial se elige Minimax o alfa-beta y la profundidad, y se muestra la acción seleccionada con sus métricas.

En la pestaña de partida se elige la profundidad y se simula la partida turno a turno.

La interfaz usa tkinter, que viene incluido en la biblioteca estándar, por lo que no requiere instalación adicional.

3.6 Pruebas automatizadas

Las pruebas se ejecutan con el siguiente comando.

```
python3 -m pytest tests/ -q
```

La suite cubre búsqueda, heurísticas, estado del juego, funciones de utilidad,

algoritmos adversariales, reglas del recurso, validación de escenarios, verificación de soluciones y los casos mínimos.

4. Formato de los escenarios

Los escenarios son archivos JSON que se encuentran en la carpeta scenarios. El programa también acepta archivos externos, siempre que respeten el formato común. La carga es completa o no se aplica: si el archivo tiene algún error, se listan los problemas y no se ejecuta nada.

Los campos del formato son los siguientes.

version indica la versión del formato.

mapa.filas y mapa.columnas definen las dimensiones de la cuadrícula.

tipos_terreno es el catálogo de terrenos, con su costo y su indicador de transitabilidad.

terreno es la matriz que describe el tipo de cada celda.

bases.A y bases.B indican las posiciones de las bases de cada bando.

recurso indica la posición del recurso.

unidades es la lista de unidades, con identificador, bando, tipo y posición.

turno indica el bando al que le corresponde actuar.

juego.portador_recurso identifica a la unidad que transporta el recurso, o es nulo si no lo lleva ninguna.

juego.profundidad_maxima_minimax indica la profundidad por defecto del árbol adversarial.

prueba describe la operación que se quiere ejecutar, con su modo y sus parámetros.

5. Interpretación de los resultados

En una prueba de búsqueda, la salida muestra primero el mapa en la consola y después un bloque con los resultados.

El campo exito indica si se encontró solución.

El campo camino contiene la secuencia de posiciones desde el inicio hasta la meta.

El campo movimientos es la cantidad de pasos del camino.

El campo costo es la suma del costo de los terrenos por los que pasa la ruta.

El campo generados es la cantidad de estados creados durante la búsqueda.

El campo expandidos es la cantidad de estados que se extrajeron de la frontera y se procesaron.

El campo maximo_frontera es la mayor cantidad de estados que estuvieron pendientes al mismo tiempo.

El campo tiempo_ms es el tiempo de ejecución en milisegundos.

Es útil distinguir entre movimientos y costo. BFS optimiza la cantidad de movimientos, mientras que UCS y A* optimizan el costo. Por eso dos algoritmos pueden devolver caminos de distinta longitud y distinto precio para el mismo problema.

Al final de cada búsqueda aparece la verificación independiente, que comprueba cuatro condiciones: que el camino sea conexo, que el costo reportado coincida con la suma real, que se haya alcanzado el objetivo y que no se hayan atravesado celdas no transitables. Si las cuatro se cumplen, el resultado se marca como válido.

En una prueba adversarial, la salida muestra la acción elegida, el valor obtenido, la cantidad de nodos generados, la cantidad de nodos evaluados, la cantidad de nodos podados y la profundidad efectiva alcanzada.

En una prueba de partida, la salida muestra la secuencia de turnos, con la acción de

cada bando y sus métricas, y al final el ganador o el motivo por el cual terminó la partida.

6. Verificación de soluciones

El verificador comprueba que el camino sea conexo, que no atraviese obstáculos, que el costo reportado coincida con la suma de las transiciones y que el último estado satisfaga la condición del objetivo.

El punto de entrada invoca este verificador automáticamente en el modo de búsqueda, de manera que cada resultado se acompaña de su comprobación.

7. Algoritmos incluidos

Búsqueda no informada: búsqueda en anchura y búsqueda en profundidad.

Búsqueda basada en costos: búsqueda de costo uniforme.

Búsqueda informada: A* con las heurísticas de Manhattan, Manhattan ponderada por costo mínimo y distancia real calculada con un recorrido en anchura.

Búsqueda con memoria limitada: búsqueda en haz, con ancho configurable.

Búsqueda adversarial: Minimax y Minimax con poda alfa-beta, con profundidad configurable.

8. Autor

Cristian Camilo Osorio Granada

Universidad de Caldas

Sistemas Inteligentes I